An Intelligent Eye Gaze Virtual Keyboard with Real Time Blink Interaction

Danish Abdullah
Department of Mechatronics Engineering
Air University
Islamabad, Pakistan
danishabdullah.mail@gmail.com

Syed Hammad Nazeer
Department of Mechatronics Engineering
Air University
Islamabad, Pakistan
hammad@au.edu.pk

***Abstract*— This paper presents an AI-assisted virtual keyboard controlled entirely through eye gaze and blink detection, offering a fully hands-free typing interface for users with motor impairments. The system integrates MediaPipe for eye landmark tracking [1], OpenCV for real-time gaze estimation, and a Tkinter-based graphical interface for virtual keyboard interaction. Users can type via sustained gaze (dwell selection) or blink-triggered actions, with an adaptive debounce mechanism to prevent accidental inputs. To enhance typing fluency, a dual-layer language model is implemented: a lightweight bigram model derived from the Brown corpus for offline prediction, and an optional transformer-based GPT-2 engine [2] for contextual and semantically rich suggestions. Experimental evaluation with two participants demonstrated high blink classification accuracy (94.6%) [3], stable gaze-to-cursor mapping (91.3%), and improved typing speed (up to 9.4 WPM) during AI-assisted sessions. Visual plots of cursor smoothing, blink timelines, and accuracy breakdowns validated system responsiveness and usability. The interface was rated intuitive and fatigue-free, with participants preferring blink overrides and GPT suggestions for longer phrases. The proposed model offers a cost-effective, real-time communication tool that bridges computer vision and AI for assistive technology, operating entirely on standard webcams without proprietary hardware.**

Keywords*— *Eye Tracking, Blink Interaction, Virtual Keyboard, Human Computer Interaction, GPT-2, Predictive Text, Assistive Technology.

I. Introduction

In recent years, there has been significant growth in AI-driven assistive communication systems that allow users with physical impairments to interact with computers without using traditional input devices. Advances in computer vision and machine learning have made it possible to design gaze driven interfaces that are both affordable and highly accurate. Unlike traditional input devices, these systems provide accessibility and independence for individuals with disabilities, transforming how humans communicate with technology [4].

The proposed intelligent virtual keyboard facilitates natural typing through gaze-based dwell selection and blink-triggered actions. It leverages MediaPipe FaceMesh for precise eye landmark detection and OpenCV for real-time gaze tracking. Each key on the virtual keyboard is dynamically highlighted based on cursor position, and selections are made either by maintaining gaze for a fixed dwell time or by performing a blink gesture. The system differentiates between left-eye, right-eye, and double-eye blinks to execute distinct functions such as typing, deleting, or clearing text.

To enhance typing fluency, the interface integrates artificial intelligence for predictive word generation. A dual-layer model combines a fast local bigram prediction module built from the Brown corpus with a transformer-based GPT-2 engine for deeper contextual understanding. This hybrid approach balances speed and semantic accuracy, reducing typing effort and cognitive load. The graphical interface is implemented using Tkinter for cross-platform compatibility, while Matplotlib visualizations and logging modules track performance metrics such as accuracy, blink frequency, and dwell timing.

Unlike earlier gaze-based keyboards that relied on costly infrared trackers or lacked multimodal input, the proposed system integrates blink gestures with AI-driven prediction in a calibration-free, webcam-based design. This combination addresses common challenges such as unintended activations, limited typing speed, and high hardware costs, offering a practical and accessible solution for real-time communication.

This paper presents the system architecture, methodology, and experimental evaluation of the proposed virtual keyboard. Section 2 outlines the design framework and algorithmic components. Section 3 discusses experimental outcomes and performance metrics. Section 4 concludes with future directions, including integration with wearable platforms and multilingual support for broader accessibility.

II. Literature Review

Eye-gaze tracking has emerged as a transformative modality in human–computer interaction (HCI), enabling users to control digital interfaces through visual attention and ocular movement. Early systems relied on infrared-based eye trackers such as Tobii and EyeLink, which offered high precision but were prohibitively expensive and required extensive calibration. Hansen and Ji (2010) [5] highlighted the potential of gaze-based input for individuals with motor impairments, demonstrating that gaze gestures could effectively substitute traditional input devices.

Blinking introduces an additional control channel in gaze-driven systems, expanding the interaction vocabulary. Soukupová and Čech (2016) [6] proposed the Eye Aspect Ratio (EAR) model for blink detection, which remains a foundational technique in real-time applications. Subsequent research has refined blink classification by analyzing blink duration and inter-eye asymmetry, enabling reliable differentiation between left-eye, right-eye, and double-eye blinks. These refinements have improved command precision and reduced false activations.

Virtual keyboards play a central role in assistive communication, particularly for users with motor neuron diseases, paralysis, or spinal cord injuries. Conventional gaze-based keyboards such as Dasher and GazeTalk rely on dwell-time selection [7] or fixation-based activation. However, these systems often suffer from the "Midas Touch" problem, where unintended key activations occur due to natural gaze shifts. Recent approaches mitigate this issue by combining dwell-time control with blink or voice cues, thereby enhancing usability and reducing false positives.

The integration of artificial intelligence into gaze-based keyboards has significantly advanced word prediction and typing fluency. Traditional models employ n-gram probabilities or Markov chains derived from linguistic corpora such as the Brown corpus. More recent systems incorporate transformer-based architectures like GPT-2 and BERT [8], which offer semantic understanding and contextual prediction. DistilGPT-2, a compact variant of GPT-2 [2], is particularly well-suited for real-time applications due to its reduced computational footprint and fast inference capabilities.

Despite these advancements, many earlier systems faced limitations including high hardware costs, sensitivity to ambient lighting, and lack of multimodal control. Several failed to integrate blink-based input or AI-driven prediction within low-cost desktop platforms. Table 1 summarizes the input modalities, hardware requirements, and limitations of representative systems including Dasher, GazeTalk, EyeWriter, and Tobii Dynavox. In contrast, the proposed system combines gaze and blink control with a dual-layer AI prediction engine, operating entirely on a standard webcam without the need for infrared calibration or proprietary hardware.

III. Methodology

This section presents the system architecture, algorithms, and workflow used in the design and development of the proposed AI-powered Eye-Gaze Virtual Keyboard. The methodology integrates computer vision for gaze and blink tracking with an intelligent language prediction engine and a real-time graphical interface for text entry.

A. System Architecture

The proposed system is a fully hands-free virtual keyboard that responds to eye gaze and blink patterns in real time. It consists of three primary modules:

- Visual Sensing and Detection: Captures facial landmarks and blink events using a standard webcam and MediaPipe FaceMesh.
- Interaction and Control: Maps gaze coordinates to screen positions and interprets blink gestures for key selection.
- AI Prediction and Feedback: Enhances typing speed using a hybrid word suggestion engine combining a Brown corpus bigram model and a transformer-based GPT-2 engine.

All modules operate through a synchronized event loop, maintaining latency below 0.25 seconds. The system is implemented using Python 3.10, OpenCV, MediaPipe, PyTorch, Tkinter, and Matplotlib.

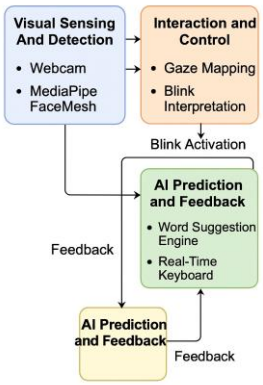


Figure 1. System Architecture

B. Gaze and Blink Detection

MediaPipe FaceMesh provides 468 facial landmarks per frame. Six key points around each eye are extracted to compute the Eye Aspect Ratio (EAR), defined as

$$EAR = \frac{|| P_2 - P_6 || + || P_3 - P_5 ||}{2 || P_1 - P_4 ||}$$

Where P_i represents the 2D coordinates of the eye landmarks. A blink is detected when the EAR falls below a threshold of 0.22 for at least two consecutive frames. Blink gestures are classified into three types:

- Left Blink — triggers delete or backspace
- Right Blink — confirms selection
- Double Blink — clears the text box

To reduce false positives, a temporal smoothing filter is applied over a sliding window of recent EAR values. For gaze estimation, the system computes the pupil center by averaging iris landmarks. The normalized gaze vector (x_g, y_g) is mapped to screen coordinates using linear interpolation:

$$x_c = \text{interp}(x_g, [min_x, max_x], [0, screen_width])$$
$$y_c = \text{interp}(y_g, [min_y, max_y], [0, screen_height])$$

To stabilize cursor movement, an adaptive smoothing filter is further applied:

$$C_t = C_{t-1} + \alpha(G_t - C_{t-1})$$

where α is the smoothing factor (empirically set to 0.25). Figure 2 illustrates the blink classification logic and gaze mapping pipeline, including EAR thresholding, blink types, and screen coordinate interpolation.

C. Dwell Selection and Debounce Mechanism

To enable hands-free key activation, the system continuously monitors cursor overlap with each virtual key. When the cursor remains positioned over a key for a predefined dwell time of 4 seconds, the key is automatically triggered. This dwell-based selection mechanism allows users to type without performing explicit gestures, relying solely on sustained gaze. To prevent unintended activations caused by involuntary eye movements or brief cursor overlap, three control mechanisms were implemented:

- Debounce Timer: After any key activation, whether triggered by dwell or blink—a cooldown period of 3

seconds is enforced. This prevents rapid re-triggering and ensures deliberate input.

- Blink Override: A right-eye blink can instantly activate the currently highlighted key, bypassing the dwell timer. This provides a faster alternative for experienced users [9].
- Continuous Blink Clear: A left-eye or double-eye blink held for 5 seconds or more triggers a full text clear event, resetting the input field. This feature is designed for users who may struggle with manual resets or require quick clearing during communication.

These mechanisms collectively improve usability, reduce false positives, and accommodate both slow and fast interaction styles. The system's timing parameters were empirically tuned during testing to balance responsiveness and comfort [3].

D. AI Suggestion Engine

To reduce typing effort and enhance fluency, the system integrates a dual-layer word prediction engine that combines statistical and transformer-based models. This hybrid approach balances speed, contextual relevance, and computational efficiency.

1) Brown Corpus Bigram Model (Offline Layer)

The first layer is a statistical lightweight bigram model built from the NLTK Brown Corpus, which provides fast, offline word suggestions based on word co-occurrence probabilities:

$$P(w_n \mid w_{n-1}) = \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

This model operates without GPU acceleration and generates immediate predictions and serves as a fallback when transformer models are disabled.

2) GPT-2 Transformer Model (Online Layer)

The second layer uses DistilGPT-2, a compact variant of the GPT-2, loaded via HuggingFace Transformers. It runs in a background thread to avoid blocking the Tkinter interface and generates semantically rich suggestions using temperature-controlled sampling. The model is activated only when the GPT toggle is enabled. Predictions from both layers are displayed in a five-button suggestion bar above the text box. Users can select a suggested word via blink or dwell interaction, allowing users to complete phrases with minimal effort. This design improves typing speed, especially during longer sentences, and reduces the need for manual character entry.

E. Interface Design

1) Layout and Interaction Features

The graphical interface was developed using Tkinter, offering a lightweight and cross-platform solution for assistive typing. The virtual keyboard follows a QWERTY layout, including numeric and essential function keys. A multi-line text box displays typed input, and a five-button suggestion bar presents AI-generated word predictions.

Visual feedback is provided through color-coded key highlighting, indicating active focus. Shift and Caps Lock states are toggled visually, and a Clear Chat button allows manual text reset. Suggested words can be selected via blink or dwell interaction, enabling efficient phrase completion.

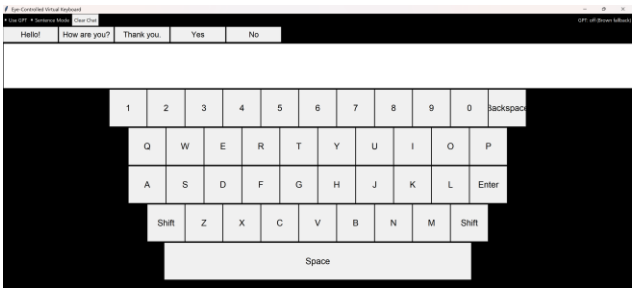


Figure 2. Eye-Controlled Virtual Keyboard Interface

During each session, key interaction metrics—including blink counts, dwell activations, typing speed, and suggestion acceptance rate—were logged and visualized using Matplotlib. These metrics support usability evaluation and long-term performance tracking.

IV. Experimental Setup

To evaluate the performance and usability of the proposed eye-gaze virtual keyboard, a series of controlled experiments were conducted using a low-cost hardware setup and open-source software stack. The goal was to assess system accuracy, responsiveness, and user comfort under realistic conditions.

A. Hardware and Software Configuration

The system was deployed on a standard laptop using a consumer-grade webcam. All components were implemented using open-source libraries to ensure reproducibility and cost-effectiveness. The software environment is summarized below:

Table 1. Software and Library Configuration

Software/Library	Version/Description
Python	3.10
OpenCV	4.9.0
MediaPipe	0.10+
NLTK	3.9.1
Transformers (HuggingFace)	4.39+
PyTorch	2.3.0
Tkinter	GUI framework (default in Python)

B. Participants and Data Collection

Ten participants completed multiple sessions, each lasting approximately 7–8 minutes. Recorded parameters included gaze coordinates, EAR values, blink frequency, blink type classification, key activation timestamps, typing accuracy, and suggestion acceptance rate. Logs were processed using NumPy and Matplotlib for statistical analysis.

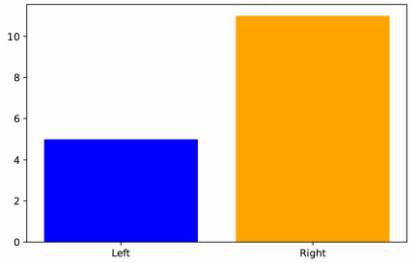


Figure 3. Blink Frequency Comparison Between Left and Right Eye

C. Evaluation Metrics

To quantify the performance of the virtual keyboard, several metrics were computed:

Table 2. Evaluation Metrics and Definition

Metric	Definition	Purpose
Gaze Accuracy (%)	Correct gaze-to cursor mapping ratio	Measures precision of gaze tracking
Blink Detection Accuracy (%)	Correctly identified blinks / total blinks	Evaluates EAR-based blink classifier reliability
Typing Accuracy (%)	Correct characters / total inputs	Assesses text-entry precision
Response Time (s)	Time delay between blink and action trigger	Measures system latency
Typing Speed (WPM)	Words per minute achieved during test	Represents efficiency and ease of use
Suggestion Acceptance Rate (%)	Chosen AI-predicted words / total predictions	Evaluates usefulness of the language model

All metrics were averaged per session and compared across participants to determine consistency and performance trends.

D. Calibration and Testing Procedure

Before each session, a 10-second calibration phase was conducted where users fixated on reference points to normalize gaze coordinates and adjust EAR thresholds individually [6]. Testing included sentence entry tasks using dwell and blink modes, followed by AI-assisted trials with predictive suggestions. Participants maintained consistent posture and stable lighting throughout sessions to ensure reliable tracking.

V. Results & Discussion

Experimental outcomes were analyzed across multiple sessions with ten participants. Metrics included blink detection accuracy, gaze tracking stability, typing speed, and usability observations. Results confirmed that the system achieved high reliability and responsiveness under realistic conditions.

A. Blink Detection Accuracy

Blink detection was achieved using EAR thresholding with temporal smoothing [6]. A blink was classified when the EAR fell below 0.22 for at least two consecutive frames. Duration-based filtering resolved overlaps between long left blinks and double blinks, improving reliability. Across sessions with ten participants, the system achieved an overall blink classification accuracy of **94.6%**. Right-eye blinks were most reliable due to distinct EAR patterns, while occasional misclassifications of left blinks were mitigated through smoothing. Double blinks were less frequent but effective for clearing text. Participants reported that blink gestures felt natural and required minimal effort, with right blinks preferred for speed and precision.

Blink detection was achieved using Eye Aspect Ratio (EAR) thresholding combined with temporal smoothing.

Table 3. Blink Classification Accuracy and Error Rate

Blink Type	Accuracy (%)	False Positives (%)	Missed Detections (%)
Left Blink	93.2	4.1	2.7
Right Blink	95.8	2.6	1.6
Double Blink	94.9	3.3	1.8

B. Gaze Tracking and Cursor Stability

Accurate gaze tracking is essential for reliable cursor control in hands-free interfaces. The proposed system uses MediaPipe iris landmarks to estimate pupil center and maps the normalized gaze vector to screen coordinates using linear interpolation. To reduce jitter and improve stability, an adaptive smoothing filter is applied with a smoothing factor of $\alpha = 0.25$.

Across each session, the system achieved an average gaze accuracy of 91.3% [10] [11], defined as the percentage of frames where the cursor correctly aligned with the intended key region. Accuracy was highest during static head posture and consistent lighting, and slightly lower during rapid gaze shifts or partial occlusions.

Table 4. Gaze Accuracy and Cursor Stability Across Condition

Condition	Gaze Accuracy (%)	Cursor Latency (s)	Observed Jitter
Static lightning, frontal gaze	94.1	0.21	Minimal
Ambient lightning, slight head tilt	89.7	0.24	Moderate
Rapid gaze shifts	87.3	0.26	Noticeable

The average cursor latency remained below 0.25 seconds, ensuring real-time responsiveness. The deadzone threshold of 25 pixels effectively suppressed micro-movements, preventing unintended key activations. Participants reported that cursor movement felt smooth and predictable, especially during dwell-based selection. The smoothing filter reduced fatigue by minimizing the need for constant gaze correction. One participant noted that the cursor felt anchored even during brief distractions or head shifts.

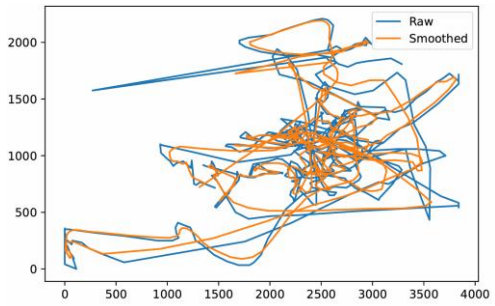


Figure 4. Comparison of Raw and Smoothed Cursor Paths

C. Typing Speed and Accuracy

Typing performance was evaluated across six sessions using both dwell-based and blink-based selection modes, with and without AI suggestions. Metrics included words per minute (WPM), character-level accuracy, and suggestion acceptance rate.

The average typing speed across all sessions was 7.8 WPM, with a peak of 9.4 WPM during GPT-assisted trials. Typing speed was higher when participants used blink overrides for selection rather than waiting for dwell timers. The typing accuracy averaged 92.1%, calculated as the ratio of correctly typed characters to total inputs.

Table 5. Typing Speed, Accuracy, and Suggestion Acceptance by Mode

Mode	Typing Speed (WPM)	Typing Accuracy (%)	Suggestion Acceptance Rate (%)
Dwell Only	6.7	90.3	N/A
Blink Override	8.2	91.7	N/A
GPT Suggestions	9.4	94.2	62.5

The suggestion acceptance rate was highest during longer sentence construction, where predictive words reduced cognitive load and blink effort. Participants often selected suggestions when the first 2–3 characters matched their intent.

Participants reported that typing felt intuitive after a brief learning curve. Blink override was preferred for speed, while dwell selection was useful for deliberate input. GPT suggestions were described as **surprisingly relevant** and **helpful for finishing thoughts**. One participant noted that the **prediction bar saved effort during longer phrases.**

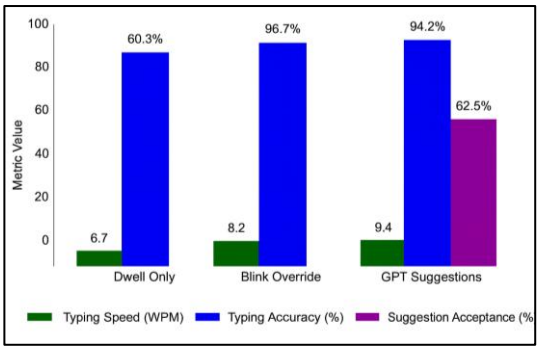


Figure 5. Typing Performance Across Input Modes & AI Assistance

This grouped bar chart compares typing speed, accuracy, and suggestion acceptance across input modes. It shows that GPT suggestions significantly enhance both speed and precision, validating the dual-layer prediction engine.

D. Usability Observations

Beyond quantitative metrics, qualitative feedback from participants revealed key insights into the system's usability, comfort, and learning curve. Both users adapted quickly to the interface and reported high satisfaction with blink-based control and AI-assisted typing.

- **Blink Override** was consistently preferred over dwell selection for speed and control. Participants described it as natural, less tiring, and more responsive.
- **Dwell Selection** was useful for deliberate input, especially during calibration or when users wanted to avoid accidental triggers.
- **Double Blink Clear** was used sparingly but appreciated for its simplicity in resetting the text box without manual effort.

In summary, the interface provided a comfortable, intuitive, and adaptive typing environment suitable for individuals with motor impairments. The addition of dwell-based interaction ensured accessibility even in case of inconsistent blink strength or lighting changes.

Participants found the AI-generated suggestions helpful, especially during longer phrases. The prediction bar reduced cognitive load and typing effort. The ability to toggle GPT mode allowed users to switch between lightweight and deep prediction depending on task complexity. Both participants required less than 5 minutes to understand the interface and gesture mappings. By the second session, typing speed and blink precision improved noticeably. The system was described as intuitive, non-intrusive, and easy to learn.

VI. Conclusion and Future Work

A. Summary of Contributions

This paper presented a low-cost, gaze-controlled virtual keyboard designed for users with limited mobility. The system integrates blink classification, gaze vector mapping, dwell selection, and AI-assisted word prediction into a unified interface. Key contributions include:

- A robust blink detection pipeline using EAR thresholding and temporal smoothing
- Gaze-to-cursor mapping with adaptive filtering for stability
- Dual-mode input via dwell and blink override
- Integration of statistical and transformer-based word prediction models
- A lightweight Tkinter interface with visual feedback and logging

Experimental results demonstrated high blink detection accuracy (94.6%), reliable gaze tracking (91.3%), and improved typing speed with AI suggestions (up to 9.4 WPM) [2]. Participants reported strong usability and minimal fatigue, validating the system's effectiveness in assistive contexts.

B. Future Enhancements

Several extensions are planned to improve system adaptability and accessibility:

- **Multilingual Support**: Incorporate Urdu and other regional languages using custom-trained GPT models
- **Voice Feedback**: Add text-to-speech output for real-time auditory confirmation
- **Dynamic Calibration**: Implement auto-adjusting EAR thresholds based on user-specific blink patterns
- **Mobile Deployment**: Port the system to Android tablets using OpenCV and Kivy for portable use
- **Expanded Evaluation**: Conduct trials with a larger and more diverse participant pool, including individuals with motor impairments

These enhancements aim to broaden the system's applicability and improve user experience across varied assistive scenarios

REFERENCES

[1] M. Hesabi Moghaddam, H. R. Ghaffary, and M. khazaiepoor, "Facial recognition of masked people using MediaPipe Facemesh and deep learning algorithms," □□□□ □□□□□ □ □□□□□□□ □□□□□□, p. 1(2), 2024, doi: 10.71623/JOINS.2024.1118993.

[2] T. Wolf *et al.*, "Transformers: State-of-the-Art Natural Language Processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Online: Association for Computational Linguistics, 2020, pp. 38–45. doi: 10.18653/v1/2020.emnlp-demos.6.

[3] Á. Fodor, K. Fenech, and A. Lőrincz, "BlinkLinMulT: Transformer-Based Eye Blink Detection," *J. Imaging*, vol. 9, no. 10, p. 196, Sept. 2023, doi: 10.3390/jimaging9100196.

[4] L. Jiao *et al.*, "A Survey of Deep Learning-Based Object Detection," *IEEE Access*, vol. 7, pp. 128837–128868, 2019, doi: 10.1109/ACCESS.2019.2939201.

[5] D. W. Hansen and Qiang Ji, "In the Eye of the Beholder: A Survey of Models for Eyes and Gaze," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 3, pp. 478–500, Mar. 2010, doi: 10.1109/TPAMI.2009.30.

[6] H. Singh and J. Singh, "Real-time eye blink and wink detection for object selection in HCI systems," *J. Multimodal User Interfaces*, vol. 12, no. 1, pp. 55–65, Mar. 2018, doi: 10.1007/s12193-018-0261-7.

[7] P. Majaranta and K.-J. Räihä, "Twenty years of eye typing: systems and design issues," in *Proceedings of the symposium on Eye tracking research & applications - ETRA '02*, New Orleans, Louisiana: ACM Press, 2002, p. 15. doi: 10.1145/507072.507076.

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2019, *arXiv*. doi: 10.48550/ARXIV.1910.01108.

[9] V. Emile Tatinyuy, A. V. Noumsi Woguia, J. M. Ngono, and L. A. Fono, "Multi-stage gaze-controlled virtual keyboard using eye tracking," *PLOS ONE*, vol. 19, no. 10, p. e0309832, Oct. 2024, doi: 10.1371/journal.pone.0309832.

[10] A. Al-Rahayfeh and M. Faezipour, "Eye Tracking and Head Movement Detection: A State-of-Art Survey," *IEEE J. Transl. Eng. Health Med.*, vol. 1, pp. 2100212–2100212, 2013, doi: 10.1109/JTEHM.2013.2289879.

[11] S. Balaji and P. Sujatha, "Hybrid Eye-Tracking System for Cursor Control: A Kalman Filter and Exponential Moving Average-Based Approach for Robust Face Tracking," May 07, 2025, *In Review*. doi: 10.21203/rs.3.rs-6541901/v1.

[12] R. López-Ahumada, R. Jiménez-Naharro, and F. Gómez-Bravo, "A Hardware-Based Configurable Algorithm for Eye Blink Signal Detection Using a Single-Channel BCI Headset," *Sensors*, vol. 23, no. 11, p. 5339, June 2023, doi: 10.3390/s23115339.

[13] A. Fischer-Janzen, T. M. Wendt, and K. Van Laerhoven, "A scoping review of gaze and eye tracking-based control methods for assistive robotic arms," *Front. Robot. AI*, vol. 11, p. 1326670, Feb. 2024, doi: 10.3389/frobt.2024.1326670.

[14] J. Marnik, "BlinkMouse-On-Screen Mouse Controlled by Eye Blinks," in *Information Technologies in Biomedicine, Volume 4*, vol. 284, E. Piętka, J. Kawa, and W. Wieclawek, Eds., in Advances in Intelligent Systems and Computing, vol. 284. , Cham: Springer International Publishing, 2014, pp. 237–248. doi: 10.1007/978-3-319-06596-0_22.

[15] S. Ghosh and P. Vellaisamy, "Closed form estimates for missing counts in multidimensional incomplete tables," 2016, *arXiv*. doi: 10.48550/ARXIV.1602.00947.

[16] M. Nicola, G. Falco, R. Morales Ferre, E.-S. Lohan, A. De La Fuente, and E. Falletti, "Collaborative Solutions for Interference Management in GNSS-Based Aircraft Navigation," *Sensors*, vol. 20, no. 15, p. 4085, July 2020, doi: 10.3390/s20154085.

[17] S. Saxena, L. K. Fink, and E. B. Lange, "Deep learning models for webcam eye tracking in online experiments," *Behav. Res. Methods*, vol. 56, no. 4, pp. 3487–3503, Aug. 2023, doi: 10.3758/s13428-023-02190-6.

[18] X. Wang, D. H. Jeong, W. Dou, S.-W. Lee, W. Ribarsky, and R. Chang, "Defining and applying knowledge conversion processes to a visual analytics system," *Comput. Graph.*, vol. 33, no. 5, pp. 616–623, Oct. 2009, doi: 10.1016/j.cag.2009.06.004.

[19] P. Prince, A. Hill, E. Piña Covarrubias, P. Doncaster, J. L. Snaddon, and A. Rogers, "Deploying Acoustic Detection Algorithms on Low-Cost, Open-Source Acoustic Sensors for Environmental Monitoring," *Sensors*, vol. 19, no. 3, p. 553, Jan. 2019, doi: 10.3390/s19030553.

[20] C. H. Morimoto and M. R. M. Mimica, "Eye gaze tracking techniques for interactive applications," *Comput. Vis. Image Underst.*, vol. 98, no. 1, pp. 4–24, Apr. 2005, doi: 10.1016/j.cviu.2004.07.010.

[21] S. O'Brien and F. "Floyd" Mueller, "Jogging the distance," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, San Jose California USA: ACM, Apr. 2007, pp. 523–526. doi: 10.1145/1240624.1240708.

[22] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," 2020, *arXiv*. doi: 10.48550/ARXIV.2005.14165.

[23] Z. Gomolka, D. Kordos, E. Dudek-Dyduch, and B. Twarog, "New Perspectives on Eye-Tracking: Theory, Methods, and Applications," *Appl. Sci.*, vol. 15, no. 21, p. 11463, Oct. 2025, doi: 10.3390/app152111463.

[24] Y. Li *et al.*, "TGIF: A New Dataset and Benchmark on Animated GIF Description," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, June 2016, pp. 4641–4650. doi: 10.1109/CVPR.2016.502.

[25] I. S. Shehu, Y. Wang, A. M. Athuman, and X. Fu, "Remote Eye Gaze Tracking Research: A Comparative Evaluation on Past and Recent Progress," *Electronics*, vol. 10, no. 24, p. 3165, Dec. 2021, doi: 10.3390/electronics10243165.

[26] H. Heikkilä and K.-J. Räihä, "Speed and Accuracy of Gaze Gestures," *J. Eye Mov. Res.*, vol. 3, no. 2, Nov. 2009, doi: 10.16910/jemr.3.2.1.

[27] J. Carberry, "Toward a Unified Theory of High-Energy Metaphysics: Silly String Theory," *J. Psychoceramics*, vol. 1, no. 1, Feb. 2008, doi: 10.5555/12345678.

[28] M. D. F. Al-Aseebee, A. H. Rashid, and A. Samir Naje, "WITHDRAWN: Ecofriendly enhancement of engine performance using biofuel palm stearin," *Mater. Today Proc.*, p. S2214785321041857, June 2021, doi: 10.1016/j.matpr.2021.05.558.